Simamba: Evaluating Simpson-Style Discretization for Mamba–3 State Space Language Models

Soumil Baldota, Ansen Shia, and David Zhang
School of Engineering and Applied Sciences
Columbia University
New York, NY, USA
{ssb2234, as8008, dwz2107}@columbia.edu

***Abstract*—State space sequence models offer linear-time alternatives to Transformer attention, but their quality and stability depend strongly on the numerical discretization used to update recurrent state. This project studies whether a Simpson-style update can improve the single-input single-output (SISO) recurrence used in Mamba–3. We implement Simamba, a Mamba–3-inspired mixer that replaces the matched trapezoid state update with a learned width-3 Simpson recurrence, and we build the end-to-end training, evaluation, checkpointing, benchmarking, and Hugging Face/vLLM export pipeline needed to test it. On a single Tesla V100, we train controlled 10M-parameter language models on SlimPajama subsets with fixed validation sampling, no-replacement training sampling, AdamW, cosine learning-rate decay, fp32 parameter storage, and gradient health logging. The final controlled runs are stable, but the current Simpson parameterization does not outperform the baselines: on the 500M-token experiment, Mamba–2 reaches validation loss 4.8625, the best Simamba Simpson-midpoint run reaches 4.9178, and the matched Simamba trapezoid run reaches 4.9326. At the 50M-token budget, trapezoid remains ahead of Simpson both without local convolution (5.8195 vs. 5.8929–5.9178) and with matched causal local mixing (5.8469 vs. 5.8793–5.8890). The systems result is similarly mixed: decode-step latency is within about 8% of Mamba–3 in the repository benchmark, but current Simamba prefill is hundreds of times slower and requires fused-kernel work. The main contribution is therefore a reproducible negative result and a diagnosis: Simpson's negative lag-2 correction is mathematically plausible but currently harder to optimize than the trapezoid update.**

***Index Terms*—state space models, Mamba, language modeling, discretization, Simpson's rule, vLLM, GPU training**

I. Introduction

Selective state space models (SSMs), especially the Mamba family, have become a practical alternative to attention for sequence modeling because they replace quadratic all-pairs attention with linear-time recurrent scans while preserving strong language-modeling quality [1], [2]. The latest Mamba–3 line extends this direction with inference-first SISO state dynamics and specialized kernels [3]. These models expose a numerical question that Transformers do not: how should the continuous-time SSM be discretized into a stable, trainable, GPU-efficient recurrence?

This project tests a simple hypothesis. Since Simpson's rule is a higher-order quadrature rule than trapezoidal integration for smooth functions, a Simpson-style state update might better approximate the forcing term in a selective SSM and improve language-model quality. We call the resulting mixer Simamba. It keeps the Mamba–3 SISO interface, projections, rotary state, skip path, and language-model wrapper, but changes the recurrence coefficients from a width-2 trapezoid update to a width-3 Simpson-inspired update.

The project evolved from an intended multi-GPU A6000 run to a single-GPU V100 study after the university Slurm cluster became unreliable. That forced the work to become more controlled and more systems-aware. We first had to fix exploding gradients, nonfinite steps, W&B logging, data preparation, checkpoint resumability, validation sampling, and GPU memory constraints. Once the training loop was stable, we ran matched ablations to separate the discretization effect from confounds such as local convolution in Mamba–2.

The result is not a positive Simamba quality claim. Instead, it is a reproducible experimental report. Simpson-style dynamics train stably after fp32 and learning-rate fixes, but the best current Simpson variants do not beat Mamba–2 or the matched Simamba trapezoid baseline. The evidence points to the learned negative lag-2 Simpson term as the likely optimization issue. This is still useful: it identifies which part of the proposed discretization needs reparameterization or annealing before larger-scale training is justified.

II. Background

A. Selective State Space Models

An SSM layer models a sequence through a recurrent state. In simplified continuous time,

$$\frac{dh(t)}{dt} = A(t)h(t) + B(t)x(t), \tag{1}$$

$$y(t) = C(t)h(t) + D(t)x(t), \tag{2}$$

where the Mamba family makes the dynamics selective by allowing input-dependent parameters [1]. Mamba–2 reformulates the computation using structured state space duality and a hardware-efficient chunk scan [2]. Mamba–3 introduces an inference-first SISO formulation that uses token-dependent query/key/value-like streams with recurrent state updates [3].

In the language models used here, the recurrent state can be written as a head-wise matrix H_t driven by a value-key outer product:

$$X_t = v_t k_t^\top, \qquad y_t = H_t q_t + D v_t, \tag{3}$$

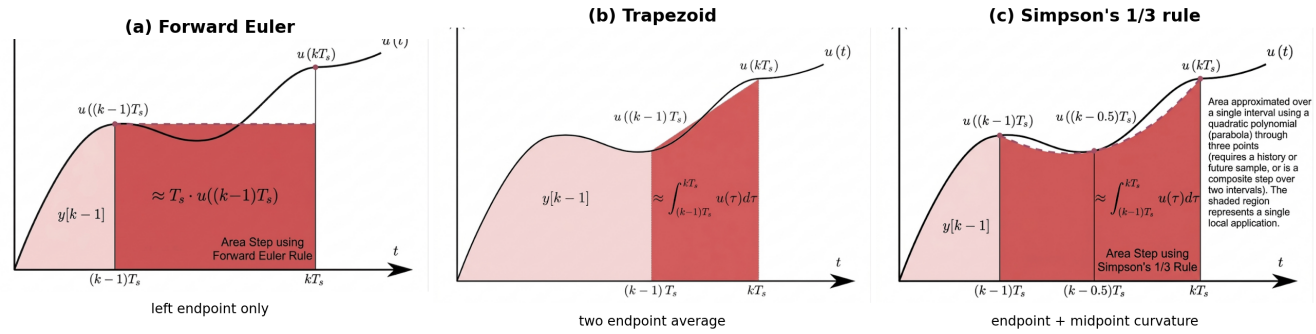

Fig. 1. Discretization intuition from the provided quadrature diagrams. Euler uses only the left endpoint, trapezoid averages the two endpoints, and Simpson's rule adds a midpoint/curvature term. Simamba uses this last idea as a causal learned width-3 SISO update, which introduces the negative lag-2 correction studied in the experiments.

where q_t, k_t, and v_t are generated by a learned input projection and rotary state. The discretization determines how X_t and previous X_{t-i} terms enter H_t.

B. Why Simpson's Rule Was Worth Testing

For a scalar linear ODE with a forcing term, the exact discrete update contains an integral of the input forcing over the interval. Trapezoidal integration approximates this integral with endpoints. Simpson's rule uses endpoints and a midpoint, and is higher order when the forcing term is smooth. The modeling hope was that an SSM recurrence with a Simpson-like width-3 correction could represent smoother continuous dynamics with fewer layers or lower discretization error.

Fig. 1 makes this quadrature view concrete. If $u(t)$ denotes the forcing term and $I_k = \int_{(k-1)T_s}^{kT_s} u(\tau)d\tau$, then forward Euler, trapezoid, and Simpson's 1/3 rule use

$$I_k^{\text{Euler}} \approx T_s u_{k-1}, \tag{4}$$

$$I_k^{\text{trap}} \approx \frac{T_s}{2}(u_{k-1} + u_k), \tag{5}$$

$$I_k^{\text{simp}} \approx \frac{T_s}{6}(u_{k-1} + 4u_{k-1/2} + u_k). \tag{6}$$

In a language-model SSM, the forcing term is not a smooth scalar but a token-dependent value-key product $v_t k_t^\top$. The Simpson panel therefore motivates, but does not fully determine, the Simamba recurrence: the implementation must remain causal and GPU-vectorizable, so the midpoint/curvature information is represented by a learned history-only width-3 correction rather than by an actual noncausal future sample.

However, this benefit is not guaranteed in a learned language model. Inputs are not smooth functions; the coefficients are data dependent; and recurrence stability matters as much as approximation order. The experiments below test the practical outcome rather than assuming the numerical-method argument transfers directly.

III. Simamba Method

A. Architecture

Fig. 2 summarizes the implemented block. The language model is the repository's MambaLMHeadModel: token IDs are embedded, each residual block applies add+RMSNorm, the selected mixer runs, and the final hidden states use a tied LM head. The Simamba mixer projects each hidden state u_t into z_t, x_t, B_t, C_t, Δ_t, A_t, a learned discretization coefficient, optional midpoint coefficient, and rotary angle parameters. It then normalizes B_t and C_t, applies the SISO recurrence, applies the D skip and optional z gate/output norm, and returns to the model dimension.

B. Matched Trapezoid Baseline

The comparison baseline is not stock Mamba–2. Mamba–2 includes causal local convolution and a different parameterization, so it is not a pure discretization control. We implemented a matched Simamba trapezoid path with the same projection layout as Simamba. Let

$$\alpha_t = \exp(A_t\Delta_t), \qquad p_t = \sigma(c_t), \tag{7}$$

where c_t is the learned trapezoid coefficient logit. The vectorized training form uses a width-2 contribution:

$$\gamma_t = \Delta_t p_t, \qquad s_t = \gamma_t + \Delta_{t+1}(1 - p_{t+1}), \tag{8}$$

and updates the recurrent state in simplified form as

$$H_t = \alpha_t H_{t-1} + s_t(v_t k_t^\top). \tag{9}$$

The implementation also includes rotary state, Q/K biases, D skip, optional z gate, and fixed boundary handling.

C. Simpson-Style Update

Simamba replaces the trapezoid contribution with a learned width-3 Simpson-like recurrence. Define

$$\alpha_t = \exp(A_t\Delta_t), \qquad \alpha_{t,1/2} = \exp(A_t\Delta_t/2), \tag{10}$$

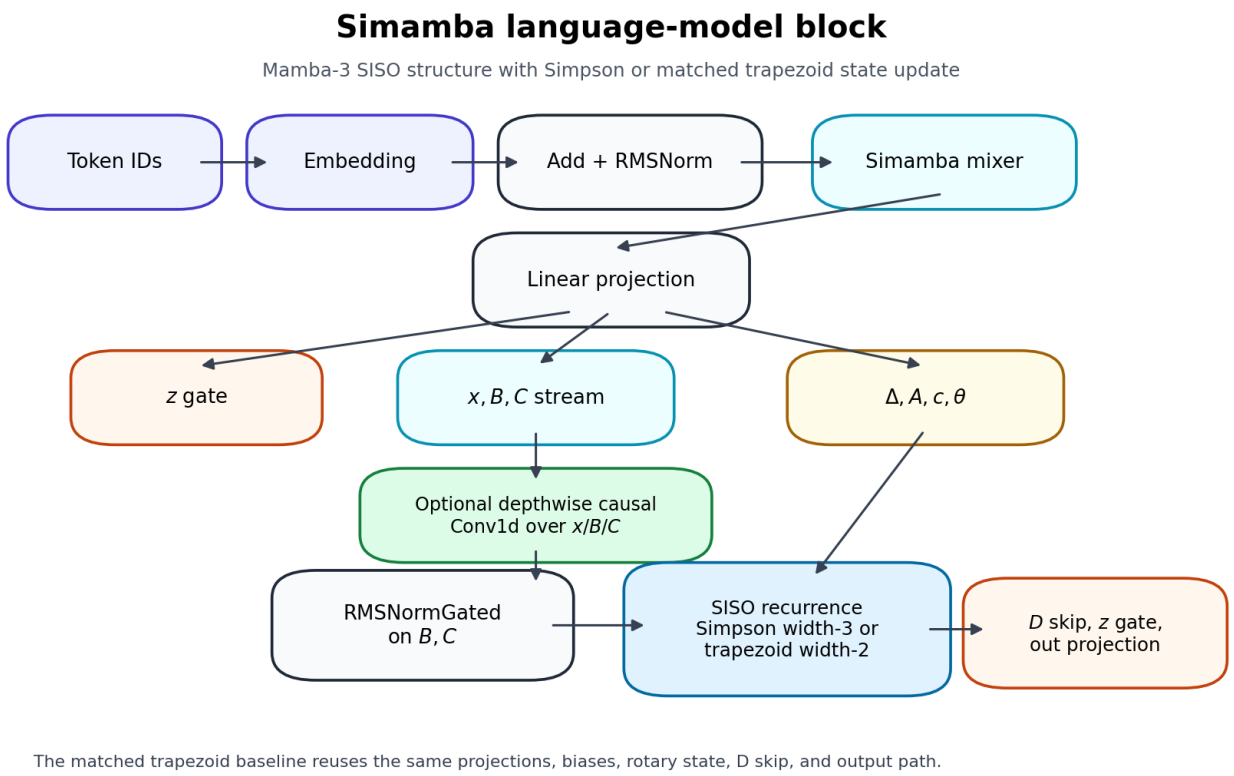

Fig. 2. Simamba architecture. The matched trapezoid baseline uses the same projections, biases, rotary state, D skip, and output path; only the SISO state-update coefficients change. Optional $d_{\text{conv}} = 4$ local mixing was added to match the Mamba–2 $x/B/C$ causal convolution confound.

and a bounded learned Simpson correction

$$s_t = \text{clip}(\sigma(c_t) m_t, 0, 1), \tag{11}$$

where $m_t = 1$ unless midpoint control is enabled, in which case $m_t = \sigma(r_t)$ is learned from an additional projection. The coefficients are

$$\gamma_{0,t} = \frac{\Delta_t}{6}\left(1 + \alpha_{t,1/2}(2 - \frac{1}{2}s_t)\right), \tag{12}$$

$$\gamma_{1,t} = \frac{\Delta_t}{6}\left(\alpha_t + \alpha_{t,1/2}(2 + s_t)\right), \tag{13}$$

$$\gamma_{2,t} = -\frac{\Delta_t}{12}\alpha_{t,1/2} s_t. \tag{14}$$

The state update is

$$\begin{aligned} H_t = \alpha_t H_{t-1} + \gamma_{0,t}(v_t k_t^\top) + \gamma_{1,t}(v_{t-1} k_{t-1}^\top) \\ + \gamma_{2,t}(v_{t-2} k_{t-2}^\top). \end{aligned} \tag{15}$$

The negative lag-2 term $\gamma_{2,t}$ is the core difference. It is also the most likely source of the noisier gradient norms observed in Fig. 6. We therefore added two controls: a coefficient logit offset to initialize s_t closer to zero, and an optional midpoint control to reduce the effective correction.

IV. Implementation

A. Code Changes

The main implementation changes are:

- `simamba.py`: Simamba mixer, Simpson/trapezoid switch, coefficient offsets, midpoint control, optional local convolution, and inference cache hooks.
- `simamba_siso_combined.py`: reference/vectorized Simpson and matched trapezoid SISO updates.
- `train_simamba_lm.py`: fixed validation spans, no-replacement epoch sampling, learning-rate decay, nonfinite-step handling, W&B logging, and checkpoint metadata.
- `prepare_slimpajama.py`: streaming SlimPajama tokenization into `uint16` memory maps.
- `eval_checkpoint_compression.py`: quantization and pruning perturbation evaluation.
- `generate_hpml_paper_assets.py`: reproducible extraction of local W&B histories and paper figures.

B. Hugging Face and vLLM Export

The best Simamba and Mamba–2 checkpoints were exported to Hugging Face. The Mamba–2 checkpoint was converted to standard `Mamba2ForCausalLM` layout with `model_type=mamba2`, making it natively recognizable by vLLM. Simamba is a custom architecture, so its repository includes `auto_map` remote code for `SimambaModel` and `SimambaForCausalLM`; vLLM can serve it through the Transformers backend with `--trust-remote-code`

`--model-impl transformers`. This preserves the architecture rather than pretending Simamba is Mamba–2.

V. Data and Training Setup

A. Dataset Preparation

We use SlimPajama [6], streamed from Hugging Face with the GPT-NeoX tokenizer. Documents are tokenized without extra special tokens, an EOS token is appended, and token IDs are stored as `uint16` memory-mapped arrays because the vocabulary fits in 16 bits. The primary dataset contains 500M train tokens and 50M validation tokens:

Split	Tokens	Documents
Train	500M	691,532
Validation	50M	49,451

The memory maps are stored under `data/slimpajama_500m_50m/`. We also prepared a 100M/10M split and tiny smoke-test subsets, but the main comparisons use 500M/50M or 50M-token budgets from the same prepared data.

B. Hardware and Runtime

The original target was four NVIDIA RTX A6000 GPUs on a Slurm cluster. That environment was unreliable, so the final experiments were run on a single NVIDIA Tesla V100-SXM2 16GB. This shaped the experimental design: we used a 10M-parameter model for controlled ablations, sequence length 128, global batch size 64, micro-batch size 8, and gradient accumulation over 8 microsteps. The largest useful 50M-parameter attempt was dominated by compilation/runtime pressure, so it was not used as the scientific comparison.

C. Deep Learning Techniques

The training stack used the following techniques:

- AdamW with weight decay for language-model pretraining [5].
- Linear warmup followed by cosine decay. The early unstable runs used an over-aggressive schedule; adding decay and reducing peak learning rate was essential.
- Gradient accumulation to fit the effective global batch on a 16GB V100.
- Gradient clipping at 1.0, with both pre-clip and post-clip norms logged. Clipping is a guardrail, not a cure for divergence.
- fp32 parameter storage and fp32 final comparisons. This follows the Mamba observation that SSM recurrent dynamics are sensitive to low-precision parameter storage [1].
- Fixed validation sampling: every run evaluates the same validation spans, which makes curves comparable.
- No-replacement epoch sampling: training draws shuffled non-overlapping token blocks rather than sampling with replacement.
- Nonfinite-step detection, checkpoint manifests, W&B run IDs, and resume metadata for long-run reliability.
- DDP-ready initialization: the script can initialize distributed data parallel training, but the reported final experiments use world size 1 on the V100.

VI. Experiments and Results

All learning-curve figures in this section are exported from locally persisted W&B/training histories. The W&B API key was not present in the report-generation shell, so the script reconstructs the curves from the synchronized local W&B output logs and JSON metric records.

A. Initial Instability

The first failure mode was exploding or nonfinite gradients before warmup completed. The loss appeared to plateau because clipping forced updates to a bounded norm, but gradient-health logs later showed nonfinite parameters/gradients. Fig. 3 shows the representative unstable run: the pre-clip gradient norm remains large while the learning rate is still tiny, and the job terminates at the first nonfinite detection.

The fixes were fp32 parameters, cosine decay after warmup, lower peak learning rates for larger runs, nonfinite skip/abort handling, and clearer W&B/checkpoint resume metadata. These changes made the controlled fp32 runs complete with zero nonfinite skips.

B. Main 500M-Token Comparison

The primary comparison trains 10M-parameter models on one pass over 500M tokens, then continues the main variants for a second pass when useful. Model dimensions are $d_{\text{model}} = 160$, 8 layers, $d_{\text{state}} = 64$, head dimension 32, sequence length 128, and tied input/output embeddings.

The validation curves in Fig. 4 are the strongest evidence against the current Simpson formulation. Mamba–2 is the best overall model, but it is not a pure discretization baseline. More importantly, the matched Simamba trapezoid baseline is competitive with and slightly better than default Simpson, while Simpson+midpoint has a better best checkpoint but worse final validation.

C. Matched 50M-Token Discretization Ablation

To reduce runtime and isolate the discretization, we ran a 50M-token matched ablation with identical model size, data, batch size, seed, and schedule. Table II and Fig. 7 show that lowering the initial Simpson correction helps, but not enough to beat trapezoid.

D. Local-Convolution Confound

Mamba–2 includes a depthwise causal convolution over the $x/B/C$ stream before the SSM update. Simamba initially did not. This matters: removing or shrinking Mamba–2's convolution hurts quality, as shown in Table III. We therefore added `--simamba-d-conv 4` to give Simamba the same local-mixing control.

This experiment is important because it prevents a misleading Mamba–2-vs-Simamba conclusion. Some of Mamba–2's advantage comes from local mixing, not only from its state discretization. After matching that mechanism, Simpson still trails the matched trapezoid update.

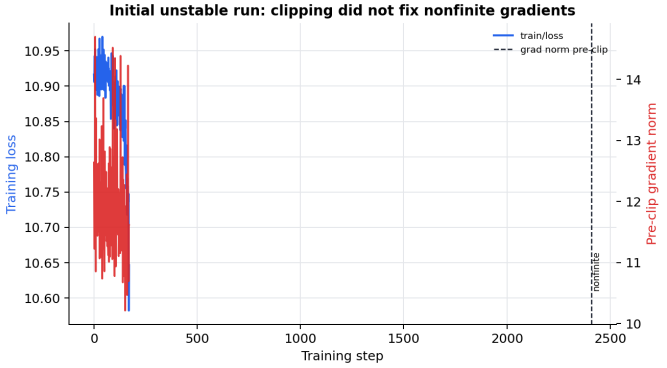

Fig. 3. Initial unstable run. Gradient clipping held the post-clip update at 1.0, but the pre-clip norm stayed large and a nonfinite gradient was detected at step 159.

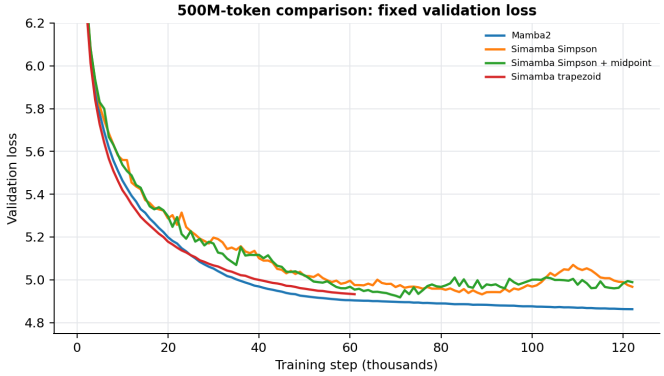

Fig. 4. Fixed validation loss on the 500M-token comparison. Mamba–2 continues improving through the second pass; Simpson variants begin to degrade on validation after their best checkpoints.

TABLE I
MAIN 500M-TOKEN LANGUAGE-MODEL COMPARISON.

Model	Best val.	Step	Last val.	Last train
Mamba–2	4.8625	122k	4.8625	4.2041
Simamba Simpson	4.9319	89k	4.9671	4.3852
Simamba Simpson+midpoint	4.9178	71k	4.9889	4.3816
Simamba trapezoid	4.9326	61k	4.9326	4.4823

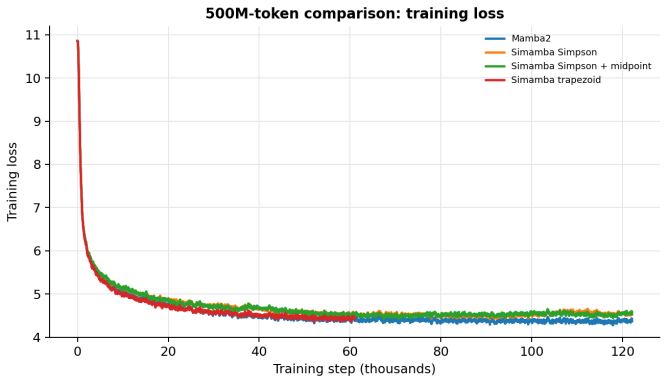

Fig. 5. Smoothed training loss for the 500M-token comparison. All models learn the training distribution, so the validation gap is not caused by a broken data path.

E. Throughput and Kernel Benchmarking

Training throughput on the V100 is shown in Fig. 9. The non-local Simpson variants are slower than Mamba–2 and trapezoid in the 500M runs, while the local-conv 50M runs are closer but still behind the fastest baselines.

The repository SISO benchmark further separates prefill and decode. The current Simamba prefill path is much slower than the Mamba–3 fused kernel path: 776.9× slower in the B1/P128/G64 case and 410.7× slower in B4/P1024/G256. Decode-step latency is much closer, about 1.08× slower. Memory is comparable at decode and can be lower for Simamba prefill in the B4/P1024 case, but the latency gap dominates.

F. Synthetic State Tracking

We also added a small synthetic classifier harness for algorithmic state-tracking tasks such as parity and modular sum. The motivation was that Simpson-style recurrences might help tasks with explicit state evolution even if the language-model result is negative. The recorded modular-sum Simamba Simpson run did not produce a useful positive result: final metrics were approximately loss 2.7811 and accuracy 0.0508 at length 128, and loss 2.7805 and accuracy 0.0542 at length 256. These tasks remain diagnostics rather than evidence for the main claim.

G. Compression and Pruning

Compression was evaluated as a side experiment using non-destructive weight perturbations on fixed validation spans. Row-wise int8 quantize-dequantize is nearly lossless for all three checkpoints, while int4 and unstructured pruning beyond 10% cause substantial degradation.

H. Overall Result

The result summary is direct: the current Simpson recurrence is trainable but not superior. The strongest Simamba Simpson checkpoint is the midpoint-control model at 4.9178 validation loss, but the best overall language model is Mamba–2 at 4.8625, and the matched trapezoid controls remain at least competitive across budgets.

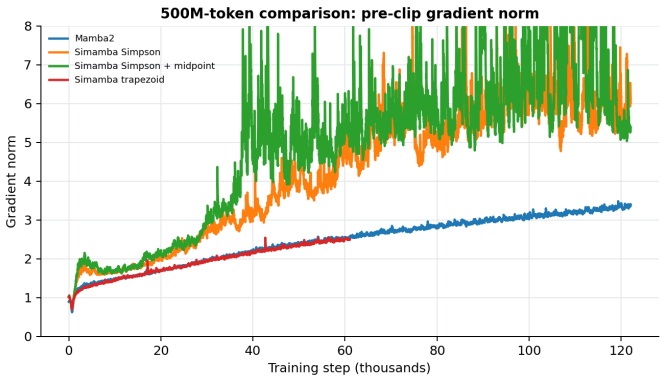

Fig. 6. Smoothed pre-clip gradient norm. The Simpson variants show larger tail gradient norms than Mamba–2 and the matched trapezoid run, consistent with harder optimization.

TABLE II
50M-TOKEN MATCHED DISCRETIZATION ABLATION.

Variant	Best/final val.	Last train	Skips
Trapezoid	5.8195	5.6111	0
Simpson default	5.9178	5.7161	0
Simpson low-control	5.8929	5.6995	0

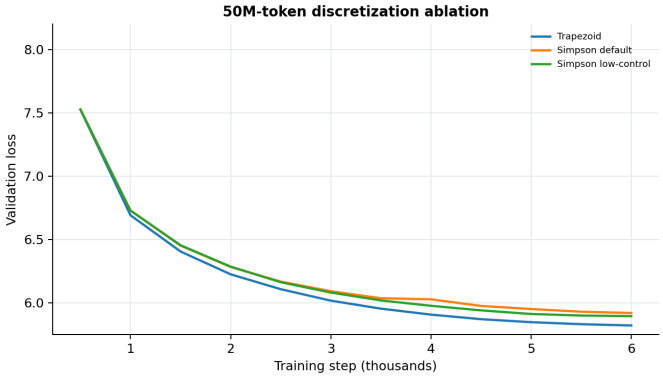

Fig. 7. 50M-token discretization ablation. The low-control initialization improves Simpson, but the matched trapezoid update remains ahead.

TABLE III
LOCAL-MIXING CONTROLS ON 50M TOKENS.

Variant	Best/final val.	Last train
Mamba–2 $d_{\text{conv}} = 2$	5.8425	5.6268
Mamba–2 $d_{\text{conv}} = 1$	5.9001	5.6958
Simamba local-conv trapezoid	5.8469	5.6309
Simamba local-conv Simpson low-control	5.8793	5.6701
Simamba local-conv Simpson	5.8890	5.6759

TABLE IV
COMPRESSION PERTURBATION LOSS DELTAS.

Checkpoint	int8	int4	10% prune	30% prune
Mamba–2 best	+0.0019	+0.5181	+0.0457	+1.4881
Simamba midpoint	+0.0014	+0.5846	+0.0433	+1.2446
Simamba trapezoid	+0.0013	+0.3771	+0.0207	+0.6680

Fig. 8. Local-mixing controls. Adding Mamba2-style local mixing narrows the gap for Simpson, but local-conv trapezoid is still better at this budget.

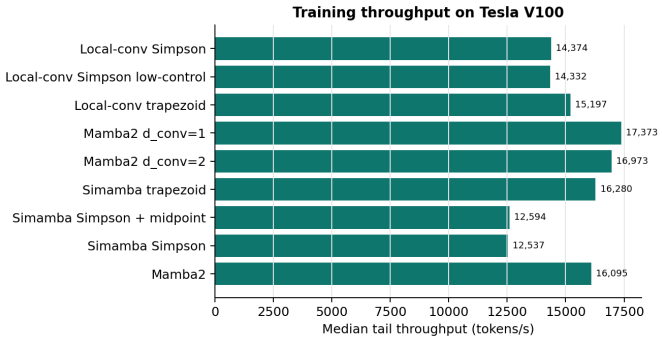

Fig. 9. Median tail training throughput on the V100, computed from the last 100 logged W&B points for each run.

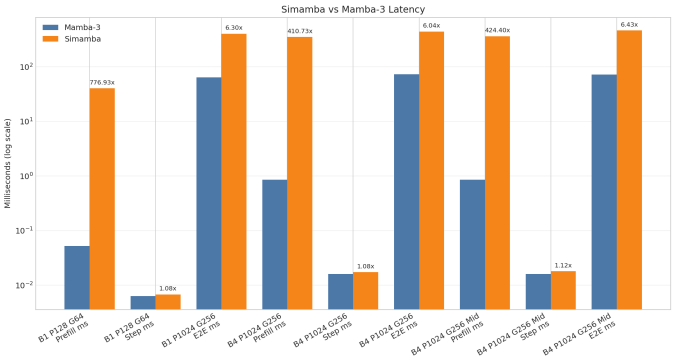

Fig. 10. Repository SISO benchmark latency. Simamba decode overhead is small, but the prefill implementation is not competitive without fused-kernel work.

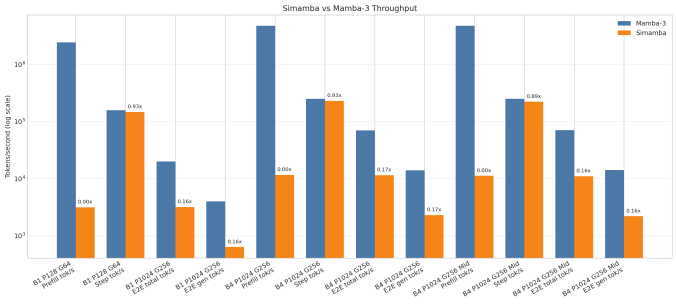

Fig. 11. Repository SISO benchmark throughput. The current prefill gap is a systems bottleneck independent of validation loss.

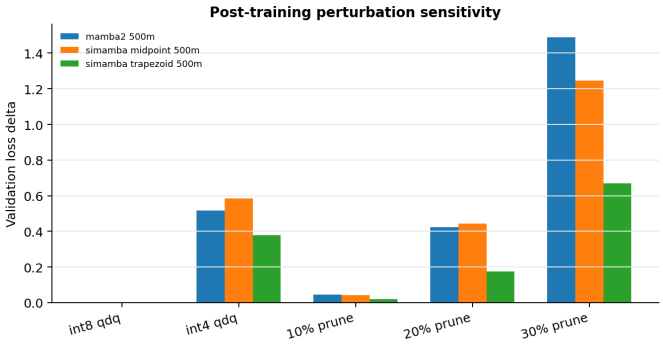

Fig. 12. Validation loss delta after post-training perturbations. Int8 row-wise quantize-dequantize is nearly lossless; int4 and larger pruning ratios require finetuning or better kernels.

VII. Discussion

A. Why Simpson Underperformed

The results suggest three interacting causes. First, Simpson's higher-order quadrature rationale assumes smooth forcing terms, but token-level language inputs are discontinuous and highly data dependent. Second, the learned negative lag-2 coefficient in (15) can produce cancellation and noisier gradients, especially early in training before the model has learned stable coefficients. Third, Mamba–2's local convolution is a major architectural confound; once we matched local mixing, Simpson improved but still did not beat trapezoid.

This does not prove that Simpson-style discretization is impossible for SSMs. It shows that the current direct parameterization is not enough. A better design may need a constrained positive decomposition, an annealed correction schedule, or a trust-region style bound on the lag-2 term.

B. Systems Lessons

The project also exposed a systems gap. Even if Simpson quality were better, the current prefill implementation would not be practical in production. The decode step is close to Mamba–3, but prefill needs a fused kernel that combines projection layout, local convolution, recurrence, and output projection without Python-level loops or excessive memory traffic. The V100 also lacks some newer Triton functionality used by the Mamba–3 kernels, so a portable fallback path is needed for older GPUs.

VIII. Future Work

The most important next steps are:

- Reparameterize the Simpson correction so the negative lag-2 term cannot dominate early training.
- Anneal the Simpson coefficient from zero after the model has learned a stable trapezoid-like recurrence.
- Build a fused local-conv Simamba kernel and a V100-compatible benchmark fallback.
- Run multi-seed 50M-token comparisons only after a single-seed Simpson variant beats the matched trapezoid baseline.
- Evaluate on longer context lengths, where higher-order state updates may have a better chance to matter.
- Add finetuning-aware int4 and pruning experiments rather than one-shot perturbations.
- Integrate the custom Simamba remote-code path more deeply into vLLM if the architecture becomes quality-positive.

IX. Conclusion

We implemented and evaluated Simamba, a Simpson-style SISO discretization for Mamba–3-inspired language models. The engineering outcome is strong: the data pipeline, training loop, validation protocol, checkpointing, W&B logging, compression evaluation, benchmarking, and model export paths are now reproducible on a single V100. The scientific outcome is negative but informative: the current Simpson recurrence does not beat Mamba–2 or the matched trapezoid baseline, and the evidence points to optimization difficulty from the negative lag-2 correction. Future work should treat Simpson as a constrained or annealed correction to a stable trapezoid recurrence rather than as an unconstrained replacement.

Artifact Availability

The best checkpoints are available as Hugging Face repositories `soumil1/simamba-midpoint-10m-slimpajama-500m` and `soumil1/mamba2-10m-slimpajama-500m`. The local paper assets are generated by `scripts/generate_hpml_paper_assets.py`.

References

[1] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv:2312.00752, 2023.

[2] T. Dao and A. Gu, "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," in *Proc. International Conference on Machine Learning*, 2024.

[3] A. Lahoti, K. Y. Li, B. Chen, C. Wang, A. Bick, J. Z. Kolter, T. Dao, and A. Gu, "Mamba-3: Improved sequence modeling using state space principles," arXiv:2603.15569, 2026.

[4] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, 2022.

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations*, 2019.

[6] Cerebras Systems, "SlimPajama: A 627B token cleaned and deduplicated version of RedPajama," Hugging Face dataset, 2023. [Online]. Available: https://huggingface.co/datasets/cerebras/SlimPajama-627B

[7] S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonell, J. Phang, M. Pieler, U. S. Prashanth, S. Purohit, L. Reynolds, J. Tow, B. Wang, and S. Weinbach, "GPT-NeoX-20B: An open-source autoregressive language model," arXiv:2204.06745, 2022.

[8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM Symposium on Operating Systems Principles*, 2023.